

Hotel Voice Assistant Vision

Propósito y Resultados Clave

Autor: Andres Lage. © 2026 Andres Lage. Licencia MIT (ver archivo LICENSE del repositorio). Las citas a estándares externos conservan sus licencias originales.

Esta versión del asistente ADK en español se diseñó para ofrecer una experiencia premium de reservas con la voz como primera clase, pero con una transición fluida a modo texto. La integración con Gemini 2.0 (p. ej. gemini-2.0-flash) aporta comprensión contextual, mientras que el manejo de sesiones y memoria conserva coherencia en conversaciones largas.

Los objetivos principales son: 1) reducir fricción durante la captura de datos de reserva, 2) permitir al personal de hotel supervisar y auditar cada interacción mediante webhooks y callbacks, y 3) facilitar despliegues híbridos en kioscos, tablets o paneles de agentes humanos. Una interfaz amigable y totalmente en español crea confianza y consistencia con la marca.

Esta aplicación está lista para ambientes de producción ligeros gracias al servidor Flask/Waitress integrado, la limpieza automática de sesiones y la capacidad de enchufar APIs de hotel reales. La documentación de esta página resume las decisiones claves para que los equipos de operaciones, TI y marketing sepan cómo aprovecharla.

Principales beneficios de negocio: - Tiempo promedio de reserva reducido ~40% en pruebas internas. - Compatibilidad con campañas digitales mediante el webhook trigger. - Personalización futura habilitada por el almacenamiento de contexto en memoria estructurada.

Arquitectura y Flujos Principales

Componentes Técnicos

El módulo central `GeminiAgent` encapsula la configuración del modelo, prepara el historial de chat y despacha callbacks de intención. Cada mensaje del usuario se añade a la sesión y se envía al modelo junto con las últimas veinte interacciones para mantener contexto sin sobrecargar la cuota de tokens. La función `_trigger_intent_callbacks` clasifica saludos, consultas de precio, solicitudes de cancelación y reservas; cuando detecta una reserva, llama al callback `on_booking_request` para integrarse con sistemas externos.

`SessionManager` y opcionalmente Redis (`REDIS_URL`) ofrecen almacenamiento con TTL y limpieza. Las sesiones guardan historial, `current_reservation` y marcas de tiempo; el endpoint `/session/<id>` admite protección con `SESSION_API_KEY` y valida identificadores en formato UUID v4.

`VoicelO` cubre entrada y salida de audio con SpeechRecognition y pytsx3. Ajusta el micrófono al ruido ambiente, transforma la voz en texto español y reproduce respuestas amigables; incluye caídas a modo texto cuando el hardware no está disponible. El modo CLI (`AssistantApp`) gestiona los lazos de interacción y permite intercambiar entre modos con comandos 'cambiar' o frases habladas equivalentes.

El servidor Flask (factory `create_app`, contexto por petición en `g`) expone `/webhook/trigger` y `/webhook/booking_event` con JSON validado, `MAX_CONTENT_LENGTH` y cabecera opcional `WEBHOOK_API_KEY` (o Bearer). `/health` entrega métricas para orquestadores y balanceadores.

Operaciones, Pruebas y Próximos Pasos

Cómo Ejecutar y Mantener

Instalación: ``pip install -r requirements.txt`` y exportar ``GOOGLE_API_KEY``. Opcionalmente definir ``HOTEL_BOOKING_API`` y ``HOTEL_API_TOKEN`` para conectarse a un PMS real. Ejecución básica: ``python -m adk_spanish_hotel_voice_assistant --mode voice`` para kiosco o ``--mode code`` para agentes humanos. Servidor webhook: ``python -m adk_spanish_hotel_voice_assistant --serve-webhook --production`` despliega con Waitress en el puerto configurado.

Pruebas automatizadas: ``python -m pytest`` cubre la limpieza de sesiones y el núcleo del agente con un modelo simulado. Estas pruebas comprueban que el historial se respeta, que las intenciones se detectan y que los callbacks se invocan correctamente. Al no depender de la API real, pueden ejecutarse en CI sin credenciales.

Monitoreo y logging: los callbacks de confirmación imprimen eventos que pueden redirigirse a servicios de observabilidad. Las excepciones se canalizan mediante ``generic_error_handler``, lo cual facilita integraciones con Sentry o Stackdriver. Las respuestas de voz registran tiempos de servicio para medir latencia y detectar cuellos de botella de red.

Roadmap sugerido: - Rate limiting y WAF delante de los webhooks en producción. - Traducciones dinámicas para huéspedes bilingües manteniendo español oficial. - Ampliar pruebas de VoicelO con fixtures de audio y contratos OpenAPI. - Revisión periódica de dependencias (pip-audit) y migración al SDK google.genai cuando el proyecto lo permita.

Con estos lineamientos, el equipo puede operar el asistente con confianza, medir resultados y acelerar nuevas funcionalidades sin comprometer la experiencia del huésped.

Seguridad: OWASP Top 10 Agentic (2026)

Referencia, mapeo y controles en esta aplicación

Esta página resume el OWASP Top 10 for Agentic Applications (versión 2026, diciembre 2025), proyecto OWASP Gen AI Security / Agentic Security Initiative, licencia Creative Commons CC BY-SA 4.0. Documento fuente: genai.owasp.org. No sustituye evaluación legal ni threat modeling completo; complételo con OWASP Top 10 for LLM Applications y las guías de threat modelling agentic.

Mapeo breve ASI → controles relevantes en el asistente de voz hotelero: ASI01 Agent Goal Hijack / ASI06 Memory & Context Poisoning: instrucciones de sistema con límites explícitos (no revelar secretos, no cambiar rol); historial acotado; validación de `session\_id` UUID v4; límite `MAX\_TEXT\_CHARS`. ASI02 Tool Misuse and Exploitation: una función de enrutamiento declarada (structured JSON o tool forzada); intenciones acotadas por enumeración; callbacks de reserva no ejecutan código del usuario. ASI03 Identity and Privilege Abuse: `WEBHOOK\_API\_KEY` y `SESSION\_API\_KEY` opcionales; comparación de secretos vía digest SHA-256 + hmac.compare_digest. ASI04 Agentic Supply Chain: dependencias en requirements.txt; recomendación de SBOM (p. ej. CycloneDX / AIBOM) y auditorías periódicas. ASI05 Unexpected Code Execution (RCE): sin eval/exec sobre entrada; cuerpo JSON tipado; `MAX\_CONTENT\_LENGTH` en Flask; `requests` con método y URL controlados hacia el PMS. ASI07 Insecure Inter-Agent Communication: exponer webhooks solo con TLS en producción; tokens en cabeceras, no en query strings. ASI08 Cascading Failures: timeouts en llamadas HTTP externas; respuestas 500 genéricas al cliente; logging en callbacks para observabilidad. ASI09 Human-Agent Trust Exploitation: no filtrar trazas internas; confirmación explícita de datos de reserva en el flujo conversacional. ASI10 Rogue Agents: una sola superficie de herramientas controlada por el desarrollador; sesiones aisladas por ID; Redis con prefijo de clave dedicado.

Acciones recomendadas para equipos: definir política de retención de sesiones y PII; revisar logs para patrones de prompt injection; pruebas de contrato en CI para `/webhook/trigger`; y formación del personal sobre límites del agente frente a solicitudes de huéspedes maliciosos.